

Aula 12: Tratamento de Exceções em Java

Programação Orientada a Objetos

Nesta aula, vamos explorar um dos mecanismos mais importantes da linguagem Java: o tratamento de exceções. Aprenderemos como identificar, capturar e tratar erros de forma elegante, tornando nossos programas mais robustos, confiáveis e fáceis de manter.

POO EM
JAVA

AULA 12

NÍVEL INICIANTE

O que são Exceções e Por que Tratá-las?

Uma **exceção** é um evento inesperado que ocorre durante a execução de um programa e interrompe o fluxo normal de instruções.

Exemplos comuns incluem divisão por zero, acesso a um índice inexistente em um array ou tentativa de abrir um arquivo que não existe.

Sem um mecanismo de tratamento, o programa simplesmente **encerra instantaneamente**, exibindo uma mensagem de erro confusa para o usuário. Com o tratamento adequado, podemos prever esses eventos e tomar ações corretivas.

```
// Sem tratamento de exceção
```

```
int[] numeros = {1, 2, 3};
```

```
System.out.println(numeros[5]);
```

```
// Resultado: programa QUEBRA!
```

```
// ArrayIndexOutOfBoundsException
```

```
// Com tratamento de exceção
```

```
try {
```

```
    System.out.println(numeros[5]);
```

```
} catch (ArrayIndexOutOfBoundsException e) {
```

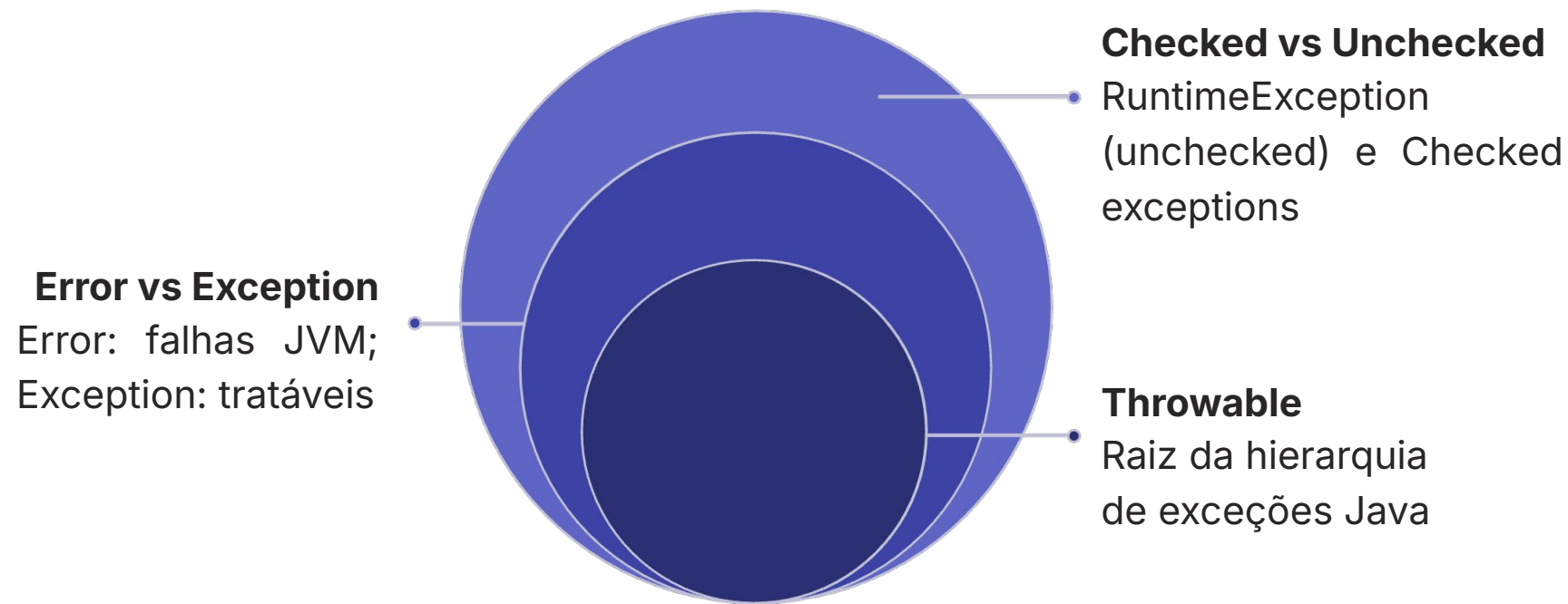
```
    System.out.println("Índice inválido!");
```

```
}
```

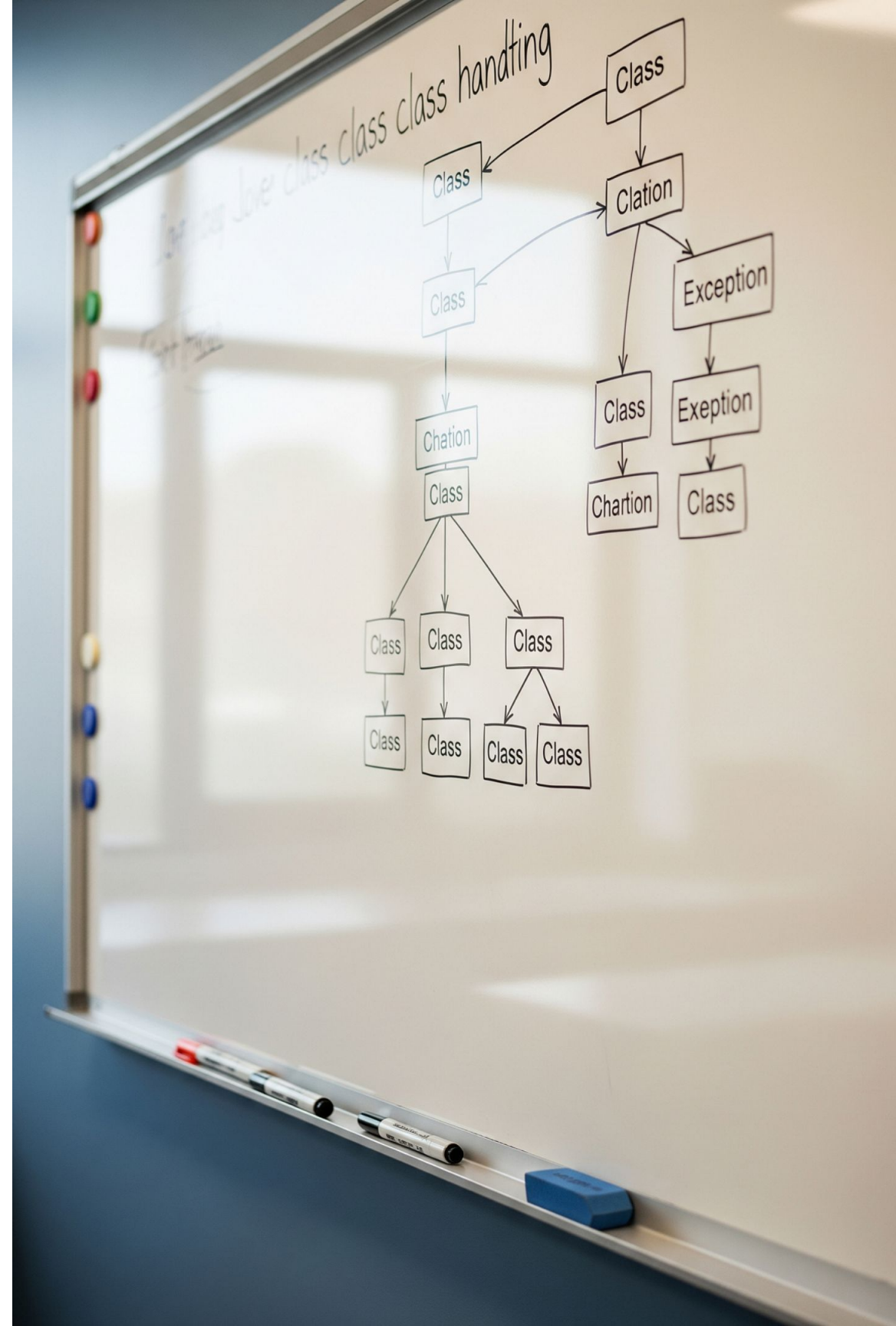
```
// Resultado: programa CONTINUA!
```

Hierarquia de Exceções em Java

Todas as exceções em Java derivam de uma única raiz: a classe `Throwable`. Compreender essa hierarquia é fundamental para tratar os erros corretamente.



A distinção entre **Error** e **Exception** é crucial: erros representam falhas graves da JVM que geralmente não devemos tentar tratar, enquanto exceções são situações que o programador pode e deve prever e tratar no código.



Checked vs. Unchecked Exceptions

✓ Checked Exceptions

São **verificadas em tempo de compilação**. O compilador exige que sejam tratadas com `try-catch` ou declaradas com `throws`. Representam situações previsíveis externas ao programa.

Exemplos: `IOException`, `SQLException`, `FileNotFoundException`

⚠ Unchecked Exceptions

São **verificadas em tempo de execução**. O compilador não obriga o tratamento. Geralmente indicam erros de lógica de programação que deveriam ser corrigidos no código.

Exemplos: `NullPointerException`,
`ArrayIndexOutOfBoundsException`, `ArithmeticException`

```
// Checked: OBRIGADO a tratar
public void lerArquivo(String nome) throws IOException {
    FileReader arquivo = new FileReader(nome);
    //compila apenas se tratar
}
```

```
// Unchecked: tratamento OPCIONAL
String texto = null;
texto.length(); // NullPointerException em runtime
```

O Java entende assim: *“Esse método pode dar um erro de entrada/saída, como arquivo não encontrado. Quem chamar esse método precisa lidar com isso.”*

No caso, `FileReader` pode gerar uma `IOException`, por exemplo, se: `lerArquivo("dados.txt");` e o arquivo `dados.txt` não existir.

Errors vs. Exceptions

Error — Não Trate!

Representam falhas **graves e irre recuperáveis** da JVM ou do ambiente de execução. Normalmente não devem ser capturados nem tratados pelo programador.

`OutOfMemoryError` — memória esgotada

`StackOverflowError` — recursão infinita

`VirtualMachineError` — falha na JVM

Exception — Trate com Cuidado

Representam condições **anormais mas recuperáveis** que o programa pode prever e tratar adequadamente, garantindo continuidade da execução.

`IOException` — erro de entrada/saída

`SQLException` — erro de banco de dados

`NumberFormatException` — conversão inválida

Resumindo:

Exception normalmente pode ser tratada pelo programador.

Error normalmente não deve ser tratada, porque indica um problema grave na JVM ou no ambiente de execução.

Bloco try-catch: Sintaxe e Funcionamento

try/catch é usado para tratar erros que podem acontecer durante a execução do programa, evitando que ele pare de funcionar de forma inesperada. O código que pode lançar uma exceção é colocado dentro do bloco `try`; o código de tratamento fica no bloco `catch`.

```
try {  
    // Código que pode gerar uma exceção  
    int resultado = 10 / 0; // ArithmeticException!  
    System.out.println("Resultado: " + resultado);  
} catch (ArithmeticException e) {  
    // Código executado SE a exceção ocorrer  
    System.out.println("Erro: " + e.getMessage());  
    // Saída: Erro: / by zero  
}  
System.out.println("Programa continua normalmente...");
```

1. Bloco try

Envolve o código arriscado. Se nenhuma exceção ocorrer, o catch é ignorado.

2. Bloco catch

Captura e trata a exceção. O parâmetro `e` contém informações do erro.

3. Fluxo após

Após o catch, o programa retoma a execução normalmente.

Múltiplos Blocos catch e Ordem de Especificidade

Um único bloco `try` pode ter **vários blocos catch**, cada um tratando um tipo diferente de exceção. A ordem importa: exceções mais **específicas** devem vir antes das mais **genéricas**, caso contrário o compilador acusará erro.

```
try {
    String[] nomes = {"Ana", "Bruno"};
    String resultado = nomes[5].toUpperCase(); // pode lançar 2 exceções!
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Índice fora do limite: " + e.getMessage());
} catch (NullPointerException e) {
    System.out.println("Referência nula encontrada!");
} catch (Exception e) {
    // GENÉRICO: deve vir por ÚLTIMO
    System.out.println("Erro inesperado: " + e.getMessage());
}
```

❏ **▲ Regra de Ouro:** Sempre coloque os `catch` do mais específico para o mais genérico. Colocar `Exception` primeiro causará erro de compilação para os catches seguintes, pois eles nunca serão alcançados.

Bloco finally

O bloco `finally` é **sempre executado**, independentemente de uma exceção ter ocorrido ou não. É usado quando precisamos garantir a execução de um código, independentemente de ter ocorrido erro ou não.

Ele é muito usado para liberar recursos, como fechar arquivos, encerrar conexões com banco de dados ou finalizar streams.

Mesmo que o bloco `catch` relance uma exceção ou que haja um `return` dentro do `try`, o bloco `finally` **garantidamente é executado**.

Fluxo:

1. **try** → tenta executar o código;
2. **catch** → trata o erro, se ele acontecer;
3. **finally** → sempre executa no final;

```
FileReader arquivo = null;
try {
    arquivo = new FileReader("dados.txt");
    // lê o arquivo...
} catch (IOException e) {
    System.out.println("Erro ao ler: "
        + e.getMessage());
} finally {
    // SEMPRE executa!
    if (arquivo != null) {
        try { arquivo.close(); }
        catch (IOException e) {}
    }
    System.out.println("Recurso liberado.");
}
```


throw vs. throws

throw — Lançar uma exceção

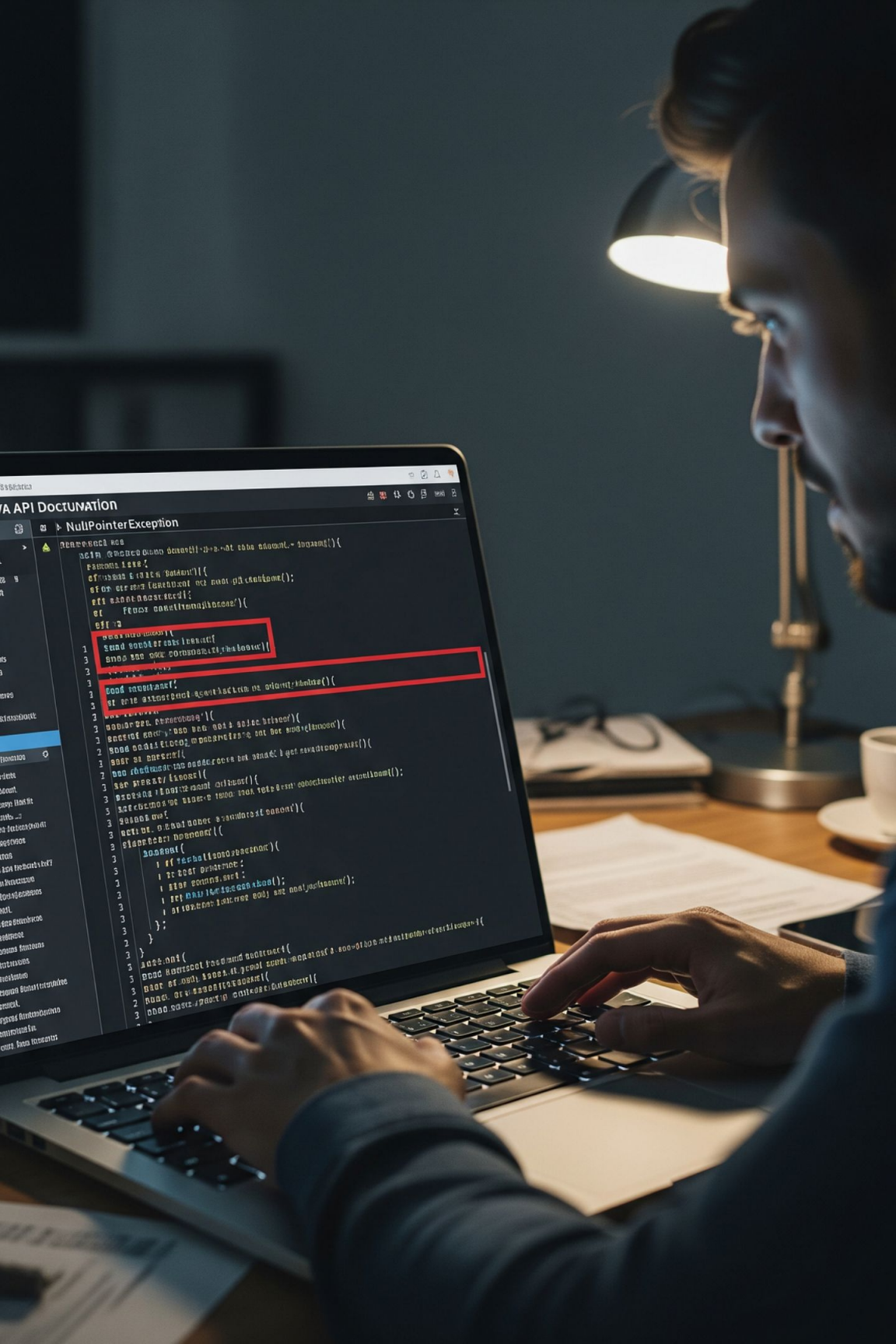
Usado **dentro do método** para lançar manualmente uma exceção específica. Seguido de uma instância de exceção.

```
public void setIdade(int idade) {  
    if (idade < 0) {  
        throw new IllegalArgumentException(  
            "Idade não pode ser negativa: "  
            + idade);  
    }  
    this.idade = idade;  
}
```

throws — Declarar exceções

Usado na **assinatura do método** para indicar que ele pode lançar uma ou mais checked exceptions, transferindo a responsabilidade ao chamador.

```
public void lerArquivo(String caminho) throws  
IOException, FileNotFoundException {  
    FileReader fr = new FileReader(caminho);  
    // leitura do arquivo...  
    fr.close();  
}
```



Exceções Comuns da API

NullPointerException

Ocorre ao tentar usar um objeto que é `null`. Ex: `String s = null;`
`s.length();` — a variável não aponta para nenhum objeto.

ArrayIndexOutOfBoundsException

Ocorre ao acessar um índice inexistente em um array. Ex: array de 3 elementos acessado com índice 5.

NumberFormatException

Ocorre ao converter uma String inválida para número.
Ex: `Integer.parseInt("abc")` lança esta exceção.

IOException / SQLException

Checked exceptions para erros de entrada/saída de arquivos e operações com banco de dados. Devem sempre ser tratadas explicitamente.

Criando Exceções Personalizadas

Podemos criar nossas próprias exceções para representar erros específicos do sistema.

Exemplo: em um sistema bancário, tentar sacar um valor maior que o saldo pode gerar uma exceção chamada: `SaldoInsuficienteException`

```
// 1. Definindo a exceção personalizada (Checked)
public class SaldoInsuficienteException extends Exception {
    private double saldo;

    public SaldoInsuficienteException(double saldo) {
        super("Saldo insuficiente. Saldo atual: R$ " + saldo);
        this.saldo = saldo;
    }

    public double getSaldo() { return saldo; }
}
```

`extends Exception` significa que a classe criada passa a ser uma exceção checked. Ou seja, quem usar essa exceção precisará tratar com `try/catch` ou repassar com `throws`.

Criando Exceções Personalizadas

Podemos criar nossas próprias exceções para representar erros específicos do sistema.

Exemplo: em um sistema bancário, tentar sacar um valor maior que o saldo pode gerar uma exceção chamada: `SaldoInsuficienteException`

```
// 1. Definindo a exceção personalizada (Checked)
public class SaldoInsuficienteException extends Exception {
    private double saldo;

    public SaldoInsuficienteException(double saldo) {
        super("Saldo insuficiente. Saldo atual: R$ " + saldo);
        this.saldo = saldo;
    }

    public double getSaldo() { return saldo; }
}
```

`extends Exception` significa que a classe criada passa a ser uma exceção checked. Ou seja, quem usar essa exceção precisará tratar com `try/catch` ou repassar com `throws`.

Criando Exceções Personalizadas

```
public class ContaBancaria {  
    private double saldo = 100.0;  
  
    public void sacar(double valor) throws SaldoInsuficienteException {  
        if (valor > saldo) {  
            throw new SaldoInsuficienteException(saldo);  
        }  
        saldo -= valor;  
        System.out.println("Saque de R$ " + valor + " realizado!");  
    }  
}
```

```
ContaBancaria conta = new ContaBancaria();  
try {  
    conta.sacar(500.0);  
} catch (SaldoInsuficienteException e) {  
    System.out.println(e.getMessage()); // Saldo insuficiente. Saldo  
    atual: R$ 100.0  
}
```


Boas Práticas de Tratamento de Exceções

Seja Específico

Capture exceções do tipo mais específico possível. Evite capturar `Exception` genérica, pois isso oculta erros inesperados e dificulta a depuração.

Use Mensagens Claras

Ao lançar exceções com `throw`, forneça mensagens descritivas que identifiquem o problema e orientem a correção, incluindo os valores que causaram o erro.

Nunca Ignore Exceções

Um bloco `catch` vazio é uma péssima prática. Sempre registre o erro no log ou tome alguma ação, mesmo que seja apenas exibir uma mensagem ao usuário.

Libere Recursos

Sempre use `finally` ou o recurso `try-with-resources` para garantir que arquivos, conexões e streams sejam fechados, mesmo em caso de erro.

try-with-resources: A Forma Moderna

Introduzido no Java 7, o **try-with-resources** fecha automaticamente qualquer recurso que implemente a interface `AutoCloseable`, eliminando a necessidade do bloco `finally` para fechar recursos.

Forma Antiga (Java 6)

```
BufferedReader br = null;
try {
    br = new BufferedReader(new FileReader
("arquivo.txt"));
    String linha = br.readLine();
    System.out.println(linha);
} catch (IOException e) {
    e.printStackTrace();
} finally {
    if (br != null) {
        try { br.close(); }
        catch (IOException e) {}
    }
}
```

Forma Moderna (Java 7+)

```
// Recurso declarado no try()
// é fechado AUTOMATICAMENTE!
try (BufferedReader br = new BufferedReader(new
FileReader("arquivo.txt"))) {
    String linha = br.readLine();
    System.out.println(linha);
} catch (IOException e) {
    e.printStackTrace();
}
// br.close() chamado automaticamente
```

Exercícios de Fixação

1

Divisão Segura

Crie um método `dividir(int a, int b)` que trate a `ArithmeticException` quando `b` for zero, retornando `0` e exibindo uma mensagem de aviso.

2

Validação de Entrada

Crie um programa que leia um número inteiro do usuário via `Scanner`. Trate a `NumberFormatException` e peça ao usuário que digite novamente enquanto a entrada for inválida.

3

Exceção Personalizada

Implemente uma classe `Produto` com o atributo `preco`. Crie a exceção `PrecoInvalidoException` e lance-a no setter quando o preço for negativo.

4

Hierarquia de Catch

Escreva um bloco `try-catch` com 3 tipos de exceções em ordem correta de especificidade. Documente por que essa ordem é necessária com comentários no código.

Resumo da Aula 12

01

Exceções = Eventos Inesperados

Interrompem o fluxo normal. Devem ser tratadas para garantir programas robustos.

02

Hierarquia: Throwable → Error / Exception

Checked (compilação) vs. Unchecked (runtime). Errors não devem ser capturados.

03

try-catch-finally

Estrutura principal. `finally` sempre executa. Use `try-with-resources` para recursos.

04

throw vs throws / Exceções Personalizadas

Lance com `throw`, declare com `throws`. Crie exceções específicas do seu domínio.

05

Boas Práticas

Seja específico, nunca ignore exceções, use mensagens claras e libere recursos corretamente.